

微网与外部电网电力调控策略

基于因果预测与两日滚动优化的购电及储能调度

摘要

针对微网在负荷、光伏与电价变化下的购电调控问题，建立历史预测、两日计划、日内调整与逐槽执行相结合的优化模型。各层明确区分已知预报和未来实际值，以跨日连续储能协调当前供需与次日需求。

问题一建立包含能量平衡、充放电效率、容量及功率界限和日循环条件的线性规划。典型日最优购电量为 59482.7 kWh，费用 35126.95 元，较无储能基线节省 26.9%。

问题二以历史实现费用标定三源凸组合预测，采用半衰期 5 天的指数加权更新。开发期选定负荷抬升系数 1.02 与光伏折减 25 kW，每天优化两日、执行首日，实际日末电量自由并跨日传递。2 月至 12 月共 334 天总费用为 1395.7 万元，其中紧急购电费 79.6 万元。固定计划的联合残差重采样显示，总成本均值 1464 万元，P95 为 1542 万元。

问题三将官方光伏预报与历史预测按 0.7 和 0.3 组合，在 6、12、18 时调整未执行计划。主方案费用 1347.1 万元，较官方预报无调整基线下降 8.01%。受控消融表明，40 情景对冲扩展反而增加 2.1 万元；冻结验证期也增加 2.30 万元，故正式方案删除对冲。

问题四将价格替换为给定波动电价，两层策略费用分别为 1459.8 和 1410.3 万元。仅用历史价格预测的扩展费用为 1465.9 和 1418.4 万元，对应额外成本 0.42% 和 0.57%。

约束残差达到浮点误差量级，跨日 SOC 连续；334 个日末电量有 330 个不同取值，未被固定。模型同时说明初始状态、单年样本与分层规划近似的适用边界，并以负结果约束复杂机制的使用，为微网购电和储能运行提供可复现的决策依据。

关键词：微网调度；两日滚动；因果预测；储能优化；日内调整；模型消融

一、 问题重述

1.1 研究背景

小区微网通过光伏发电、储能电池和外部购电共同满足负荷。电池能够将低价时段的电能转移到高价时段，也能够吸收光伏富余电量；但预测偏差会使计划与实际需求不匹配，导致额外购电或弃电。因而，调度问题不仅是寻找一条低成本的购电曲线，还要使这条曲线在信息逐步到达的条件下可执行。

1.2 四个问题的递进关系

问题一依据典型日电价、负荷与光伏预测，建立当天购电及储能调度模型，要求日初、日末电量相同，并给出指定时段结果。问题二考虑逐日变化的负荷和光伏，要求每天零点作出购电计划，实际短缺按当期五倍电价紧急购电，常规购电按计划结算。问题三允许利用零点及 6、12、18 时发布的光伏预报制定和调整计划，调整还会产生相对原计划的偏差费用，需要同时评价新增信息和调整时机的作用。问题四进一步考虑波动电价，分别重算无日内调整与有日内调整两种策略。本文在共同的储能约束下逐步改变信息集与结算目标，使四问结果具有清晰的比较基础。

二、 数据理解与总体分析

2.1 数据处理与评价区间

附件 1 给出 144 个 10 分钟时段的典型日电价、负荷和光伏；附件 2 给出 2025 年实际负荷与光伏；附件 3 给出每天四次发布的 24 小时光伏预报；附件 4 给出波动电价。本文以电量为优化变量，功率乘 $\Delta t = 1/6$ 小时后统一为 kWh，电价保持元/kWh。附件与结果模板的时间文字存在一个 10 分钟刻度差异，因此输出沿用模板标签，以行序号一一对应原始槽数据，避免因字符串匹配造成错位。本文指定时段表也沿用结果模板标签。

附件 3 中提前 k 小时的值对应发布时刻加 k 小时，故每次更新都按发布时刻重建时间轴，再插值至 10 分钟粒度。6、12、18 时预报只用于该次发布之后尚未执行的时段，不能直接套用零点预报的映射。负荷与光伏历史预测均仅使用决策日前的实际值，典型日曲线视为题目给定的外生参考资料，不从完整年度实际值重新计算。

根据结果模板，问题二至四统一评价 2025 年 2 月 1 日至 12 月 31 日，共 334 天。1 月数据用于形成滞后特征与预热权重。当前实验在 2 月 1 日以 6000 kWh 初始化储能；这是评价区间的统一初始条件，并非对 1 月运行后的储电量进行推导。题面给出 1 月 1 日初值，因此本文结果应理解为指定报送区间的条件回测，初始状态的局限在模型评价中讨论。

2.2 误差结构与决策目标

数据分析表明，光伏光照样本的预测误差随提前时间增加，近似关系为 $MAE(k) \approx 14.8 + 29.0k$ kW，拟合 $R^2 = 0.982$ 。下午 13 至 18 时的光伏 MAE 随发布时刻由零点推进至 6 时、12 时，从 413 kW 降至 248 kW、76 kW。日内误差的前四阶相关系数为 0.79、0.60、0.40、0.21，说明逐点独立扰动会破坏误差的时间结构。因此，日内更新预报具有调度价值；整日联合残差场景作为候选扩展，通过消融决定是否进入正式方案。

紧急购电与计划购电的价格不对称，最小预测均方误差并不直接等于最小购电成本。本文用历史实现费用标定预测组合，在名义预测上加入小幅保守修正，再利用当前实际状态逐槽执行。候选场景法保留负荷和光伏残差的同日联合结构，避免只考虑光伏误差而遗漏净负荷风险；若留出实验不支持，则按简约原则删除。

2.3 两日规划与滚动执行

每天零点优化当日与次日共 288 个槽，只承诺首日计划；次日计划是储能机会成本的近似参考，到次日零点会重新计算。该思路借鉴滚动时域优化 [1]，但本文采用的有限窗口与实时执行规则并不具有自动成立的无限时域最优性保证。问题三在日内预报发布时重算剩余时段，执行层每 10 分钟只读取当前实际值，日末储电量自然传递至次日。表 1 概括不同问题的信息范围。

表 1 各问题的信息集与调度结构

问题	零点负荷与光伏输入	电价输入	调整与储能结构
一	典型日给定数据	典型日给定	单日规划、日循环
二	因果历史预测	典型日电价	无日内调整、两日滚动
三	历史负荷与组合光伏预报	典型日电价	三次更新、跨日连续
四	分别沿用问题二、三预测	给定波动电价 G	分别重算两层策略
扩展 H	与问题四对应层一致	历史价格预测	评价电价信息价值

三、模型假设

1. 每 10 分钟内功率按常数近似，负荷不可主动削减。该假设使连续能量平衡转化为有限维线性约束。
2. 储能充电和放电各按 90% 效率计损，往返效率为 81%；不另计自放电与寿命衰减。该解释适用于题目给定的固定效率条件。
3. 微网不售电；无法被负荷或储能吸收的供给可弃置。预测规划中的弃光和执行中计划购电富余的弃电应区别理解，后者不获得退款。
4. 常规购电按已承诺电量结算，短缺额外以五倍当期价格采购。问题三各槽按最终有效调整量相对零点原计划结算一次，避免对同一槽重复计收偏差费用。
5. 问题一保留题面要求的日循环；问题二至四不人为固定每日实际末端电量，两日

规划最远端仅受容量边界约束。日内调整规划的末端参考由当日两日计划内生给出，执行允许偏离该参考。

6. 波动电价主分析 G 将附件 4 给定价格作为规划窗口内已知输入；H 作为仅有历史价格的扩展。G 不是对未来实际负荷或光伏已知的假设。

四、符号说明

表 2 主要符号、含义与单位

符号	含义	单位
$d, t; T, H$	日期、日内槽；日槽数 144、规划槽数 288	—
$L_{d,t}, P_{d,t}$	实际负荷、光伏电量	kWh
$\hat{L}, \hat{P}$	决策时刻可得的预测电量	kWh
$p_{d,t}$	交易电价	元/kWh
$x_{d,t}^0, x_{d,t}^a$	零点计划、各槽最终有效调整购电量	kWh
$c_{d,t}, q_{d,t}$	充入、放出的电网侧电量	kWh
$E_{d,t}$	时段末储能电量	kWh
$e_{d,t}, s_{d,t}$	紧急购电量、富余弃电量	kWh
$\eta, \bar{c}$	单向效率 0.9、单槽充放上限 5000/6	—、kWh
w_d, u_d	历史负荷、光伏的三源凸组合权重	—
h, κ, m	EWMA 半衰期、负荷抬升、光伏折减	天、—、kW
λ, S	官方光伏预报占比、情景数	—
$\mathcal{I}_{d,t}$	决策时刻已获得的信息集合	—

带上标 s 的变量表示第 s 个情景， $[z]_+ = \max(z, 0)$ 。下文未列出的局部辅助变量在首次出现时定义。

五、问题一：典型日确定性储能调度

5.1 线性规划模型

对给定的典型日，决策变量为购电 x_t 、充电 c_t 、放电 q_t 、电量 E_t 及弃光 s_t 。在不需要紧急购电的确定性条件下，目标为

$$\min C_1 = \sum_{t=1}^T p_t x_t. \quad (1)$$

每个槽满足供需平衡与储能递推：

$$P_t + x_t + q_t = L_t + c_t + s_t, \quad (2)$$

$$E_t = E_{t-1} + \eta c_t - q_t / \eta. \quad (3)$$

充电量从电网侧计量，故进入电池的电量 ηc_t ；向负荷放出 q_t 需消耗 q_t/η 。容量、功率、非负性与循环条件为

$$1200 \leq E_t \leq 10800, \quad 0 \leq c_t, q_t \leq 5000/6, \quad (4)$$

$$x_t \geq 0, \quad 0 \leq s_t \leq P_t, \quad E_0 = E_T = 6000. \quad (5)$$

所有约束与目标均为线性，共 720 个变量、289 条等式约束，采用稀疏矩阵和 HiGHS 求解。线性规划算法可高效求解该规模问题 [2]。模型未引入充放互斥二元变量，因此解后必须检查同槽同时充放；所得正式解通过该项检查。

5.2 最优策略与题目指定结果

最优全天购电量为 59482.7 kWh，购电费为 35126.95 元，较无储能基线节省 26.9%。电池利用低价时段充电，在高价及光伏不足时段释放电能，全天弃光为 0。题面指定的购电及储能结果列于表 3、表 4。图 1 中的充放电时序同时受峰谷价格和光伏出力影

表 3 问题一指定时段购电量 (kWh)

时间段	购电量
10:00-10:10	0.00
12:00-12:10	486.40
14:00-14:10	0.00
16:00-16:10	394.93
18:00-18:10	636.98
20:00-20:10	0.00
全天购电量	59482.7
全天购电费/元	35126.95

表 4 问题一指定区间储能充放电量 (kWh)

时间段	充电量	放电量
0:00-4:00	4500.00	0.00
4:00-8:00	833.33	6365.84
8:00-12:00	4787.96	1703.00
12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13
20:00-24:00	5333.33	2859.87
0:00 储电量	6000.00	
24:00 储电量	6000.00	

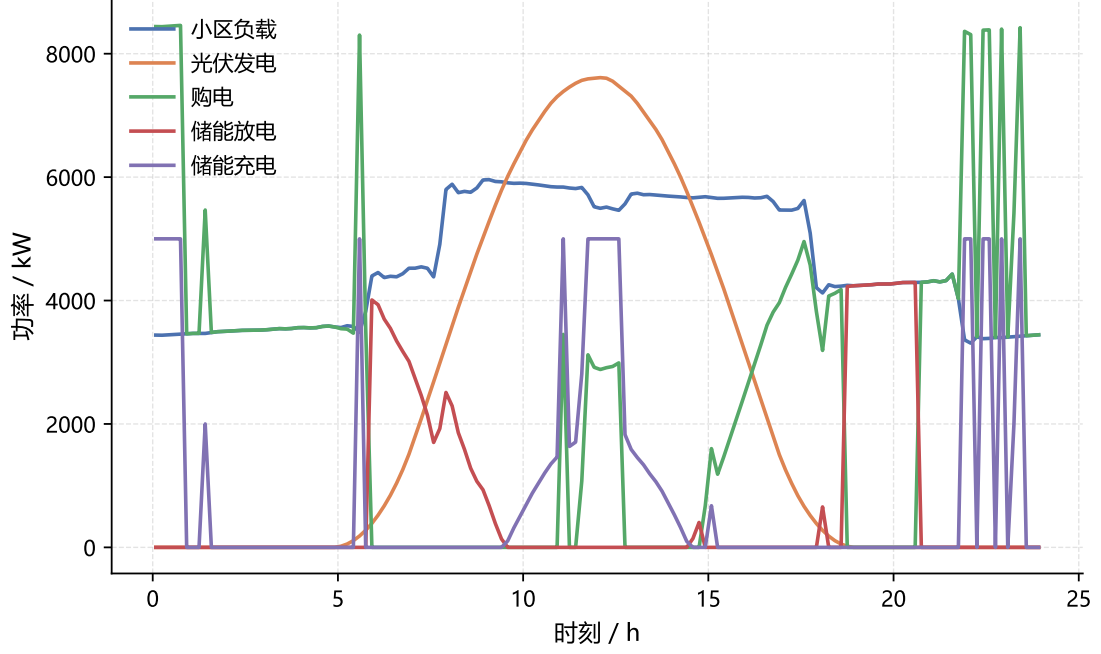


图 1 典型日购电、光伏与储能联合调度

响。由于循环条件约束日末电量，全天节省不是通过消耗初始储能获得的暂时收益。四小时汇总区间内可能同时出现正的累计充电与放电量，这表示不同 10 分钟槽采用不同动作，不意味着同槽同时充放。

本问结论：确定性典型日最优购电费为 35126.95 元，储能节省比例 26.9%；该模型提供后续各问共用的物理约束。

六、 问题二：历史预测与两日滚动购电

6.1 面向费用的历史组合预测

在零点尚不知道当天实际负荷与光伏，因此根据周周期与短期持续性构造

$$\hat{L}_{d,t} = w_{d,1}L_{d-7,t} + w_{d,2}L_{d-14,t} + w_{d,3}\bar{L}_t, \quad (6)$$

$$\hat{P}_{d,t} = u_{d,1}P_{d-1,t} + u_{d,2}P_{d-2,t} + u_{d,3}\bar{P}_t, \quad (7)$$

其中 $\bar{L}_t, \bar{P}_t$ 为题目给定典型日电量，权重非负且各自和为 1。两组三源权重在步长 0.2 的单纯形网格搜索，共 $21 \times 21 = 441$ 个组合。令 $C_j(w, u)$ 为历史日 j 使用该权重作计划并以实际值因果执行后的总费用，递推评分为

$$M_d(w, u) = \rho M_{d-1}(w, u) + (1 - \rho)C_{d-1}(w, u), \quad (8)$$

$$\rho = 2^{-1/h}, \quad (w_d, u_d) = \arg \min_{w, u} M_d(w, u).$$

历史首个可用评分初始化 M ，随后每日只吸收前一日实现费用。半衰期取 $h = 5$ 天。历史成本表采用单日规划、自由末端执行的代理费用，正式运行采用两日滚动，二者不能视为完全相同的优化目标；这种代理降低了逐日权重标定的计算量。

为控制高价短缺，对预测施加

$$\tilde{L}_{d,t} = \kappa \hat{L}_{d,t}, \quad \tilde{P}_{d,t} = [\hat{P}_{d,t} - m\Delta t]_+, \quad \kappa = 1.02, \quad m = 25 \text{ kW}. \quad (9)$$

若忽略储能与跨期耦合，仅考虑计划量 z 和随机净需求 N ，最小化 $pz + 5p\mathbb{E}[(N - z)_+]$ 给出 $F_N(z) = 0.8$ 。这解释了保守修正的方向，但不能据此直接确定含储能模型的最优裕度；具体参数仍由历史开发期费用选择。

6.2 两日计划及次日预测

每日以实际日初电量 $E_{d,0}$ 初始化 288 槽规划，目标为

$$\min \sum_{\tau=1}^{288} \hat{p}_{\tau} x_{\tau} + \varepsilon \sum_{\tau=1}^{288} (c_{\tau} + q_{\tau}), \quad (10)$$

并对预测负荷、光伏施加式 (2)–(4)。微小吞吐量正则项用于减少无意义充放循环，报送购电费不含此数值求解罚项。当日末 E_{144} 和窗口最远端 E_{288} 均不设置等式终值，仅满足容量界限。次日负荷利用 $d+1-7, d+1-14$ 的已发生曲线，当日冻结的光伏权重应用于截至 $d-1$ 的最近两条实际曲线；禁止使用当天尚未发生的光伏来构造次日预测。

6.3 逐槽因果执行

固定当前有效承诺量 x_t ，观察当前实际负荷和光伏，记 $n_t = L_t - P_t - x_t$ 。当 $n_t \geq 0$ 时，

$$q_t = \min\{n_t, \bar{c}, \eta(E_{t-1} - 1200)\}, \quad e_t = n_t - q_t, \quad c_t = s_t = 0. \quad (11)$$

当 $n_t < 0$ 时，

$$c_t = \min\{-n_t, \bar{c}, (10800 - E_{t-1})/\eta\}, \quad s_t = -n_t - c_t, \quad q_t = e_t = 0. \quad (12)$$

然后按式 (3) 更新储电量。这一规则只依赖当前槽及此前信息，不为追赶日末目标而额外紧急购电充能。次日初值为本日实际末值：

$$E_{d+1,0} = E_{d,T}^{\text{exec}}. \quad (13)$$

计划富余不会退费，故实现费用为

$$C_2 = \sum_d \sum_t p_t x_{d,t}^0 + 5 \sum_d \sum_t p_t e_{d,t}. \quad (14)$$

6.4 时间留出选择与总费用

2 月至 6 月为开发期，7 月至 12 月为验证期；后者只报告性能，不用于重新选择全局参数。表 5 显示相邻候选表现接近。所选 $m = 25$ 在开发期最优，在验证期排名第 5； $m = 50$ 的全年描述值略低，也不能据此倒过来改选。因此 334 天汇总包括开发期，不能称为全区间独立样本外测试。正式结果总费用 1395.7 万元，其中计划购电费 1316.0 万元、紧急购电费 79.6 万元；分项独立舍入可能与总额相差 0.1 万元。图 2 给出储能轨迹，日末值随供需变化而变化，验证了跨日状态连续与非固定末端条件。

表 5 问题二部分候选的时间留出结果（万元，均为 $h = 5$ ）

κ	m/kW	开发期	验证期	全区间	验证排名
1.02	25（选定）	574.0	821.6	1395.7	5
1.02	50	574.2	820.6	1394.8	4
1.015	75	574.3	821.7	1396.1	6
1.02	75	575.0	820.3	1395.3	2

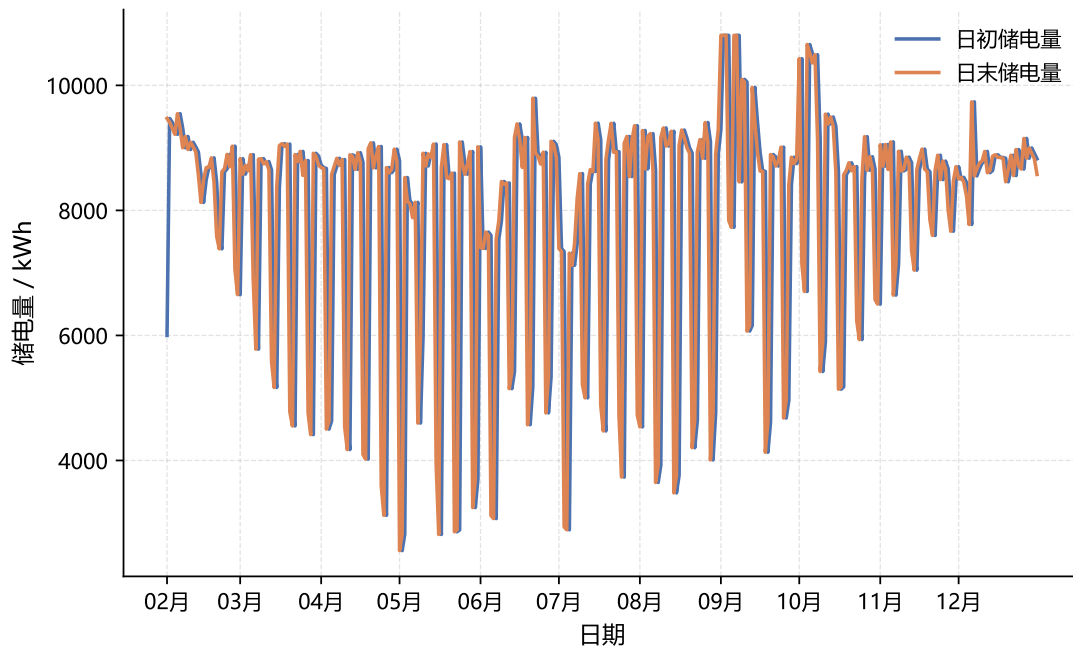


图 2 问题二实际储能电量及日末电量变化

6.5 指定日期结果与年度风险

四个指定日期的计划购电、储能与紧急购电结果见表 6–8。计划购电费与包含紧急费的总费用分别列示，以保持题目结算含义。固定已生成的两日滚动计划，对负荷和光伏联合残差作同月整日块重采样，得到 100 个模拟评价年度。这种模拟是条件于固定计划的离线压力评估，不是重新训练并逐年适应的独立样本外试验。总成本均值 1464 万元，标准差 44.53 万元，P95 为 1542 万元，尾部均值 CVaR95 为 1566 万元。CVaR 刻画超过分位阈值的平均损失 [3]，适合补充均值指标。图 3 表明实际评价区间的 1395.7 万元低于重采样均值，不能把这一实现值直接等同于未来期望成本。

本问结论：基于开发期选择的历史预测与两日滚动策略在 334 天内产生 1395.7 万元费用；实际末端电量自由，风险评价仍显示明显的高费用尾部。

表 6 问题二指定日期计划购电量与购电费

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00-10:10	0.00	0.00	0.00	0.00
12:00-12:10	529.03	0.00	553.22	975.48
14:00-14:10	0.00	0.00	0.00	0.00
16:00-16:10	428.10	169.90	464.46	796.72
18:00-18:10	744.10	534.28	802.45	731.57
20:00-20:10	0.00	0.00	0.00	0.00
全天计划电量/kWh	62281.59	42865.34	69038.87	96267.73
全天计划购电费/元	38733.07	24447.24	42750.14	61810.51
紧急购电费/元	1592.53	0.00	1003.07	83.06
含紧急费总额/元	40325.60	24447.24	43753.21	61893.57

表 7 问题二指定日期储能充放电与首末电量 (kWh)

区间	3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00-4:00	1376.88	122.05	447.73	200.31	2042.44	43.58	2040.21	276.70
4:00-8:00	992.89	4988.27	593.50	1782.09	405.75	6608.26	1608.84	1579.87
8:00-12:00	5356.38	2777.38	7656.06	0.00	5589.93	1896.62	5310.48	7513.53
12:00-16:00	4272.48	534.09	0.00	0.00	5123.65	597.06	7804.74	2276.46
16:00-20:00	313.91	5924.64	242.20	5524.96	213.59	5649.98	0.00	5685.08
20:00-24:00	4089.80	2792.53	8635.18	2570.34	8474.10	2826.51	8405.16	2848.38
0:00 电量	9061.33		5175.11		8709.06		8652.78	
24:00 电量	4780.15		9794.87		8793.57		8883.03	

表 8 问题二指定日期全部紧急购电事件

日期	紧急购电时段	紧急购电量/kWh
03 月 20 日	20:30-20:40	228.2867
06 月 21 日	无	0.0000
09 月 23 日	20:20-20:40	103.1061
09 月 23 日	20:50-21:10	43.2148
09 月 23 日	21:20-21:40	17.9760
12 月 21 日	7:50-8:00	14.2992

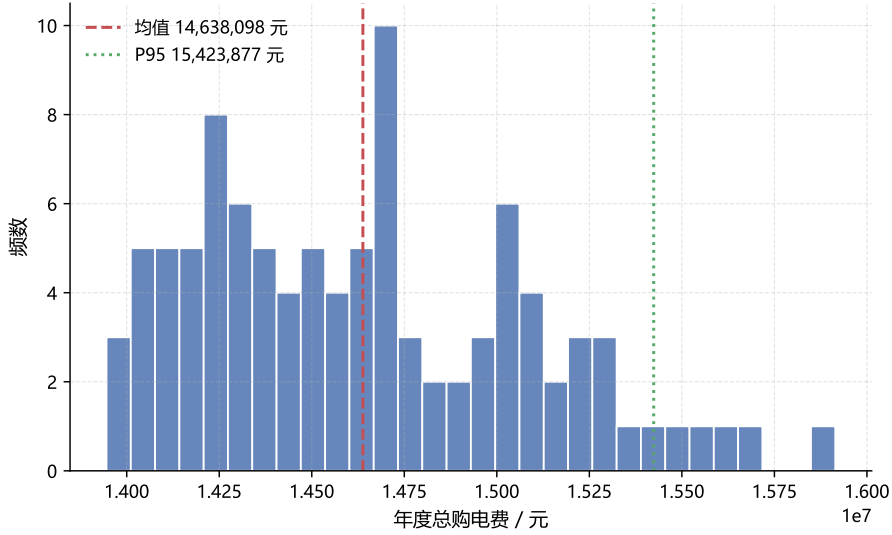


图 3 固定计划下联合残差重采样的总费用分布

七、问题三：组合预报与日内调整

7.1 统一发布相位的组合预报

零点计划层以及各调整时刻 $r \in \{6, 12, 18\}$ 均采用

$$\tilde{P}_{d,t}^{(r)} = \left[\lambda \hat{P}_{d,t}^{\text{off},r} + (1 - \lambda) \hat{P}_{d,t}^{\text{hist}} - m \Delta t \right]_+, \quad \lambda = 0.7. \quad (15)$$

官方预报使用对应发布时刻的曲线，历史项与问题二一致；负荷继续使用因果历史预测和 $\kappa = 1.02$ 修正。第二日尚无对应零点官方预报，使用零点可得的历史外推。采用相同组合形式的目的，是使日内更新仅反映新信息，而不同时改变预测构造。

7.2 调整费用的等价表达

对某个尚未执行的槽，设原计划为 x^0 、最终调整量为 x^a 。当 $x^a \leq x^0$ ，购电加违约费用为 $px^a + 0.5p(x^0 - x^a)$ ；当 $x^a > x^0$ ，费用为 $px^0 + 1.5p(x^a - x^0)$ 。二者统一为

$$g(x^a; x^0) = px^a + 0.5p|x^a - x^0|. \quad (16)$$

因此不能将原计划费用再完整相加，否则会重复计费。引入 $y \leq x^0$, $y \leq x^a$, $y \geq 0$ ，则 $g = 1.5px^a - py + 0.5px^0$ 在最优时成立。最后一项对调整决策是常数，可在求解时省略，但费用统计必须恢复。

7.3 名义调整与日末参考

每次新预报发布后，以当前实际 SOC 初始化剩余时段规划，已执行时段不再修改。零点两日规划产生当日末的参考电量 $E_{d,T}^{\text{ref}} = E_{d,144}^{\text{plan}}$ 。当前日内调整 LP 保留

$$E_{d,T}^{\text{adj}} = E_{d,T}^{\text{ref}} \quad (17)$$

作为规划连接条件；该参考由每日预测、价格与日初状态共同决定，并非恒定 6000 kWh。实际执行仍用式 (11)–(12)，不强制到达此参考。因而必须区分“规划采用动态日末参

考”和“实际日末电量自由”。这种分层连接保留了次日储能需求的部分信息，但日内层尚未直接重算完整两日窗口，属于模型近似。

7.4 联合场景候选扩展

为检验随机规划能否进一步降低短缺风险，先构造一个对冲候选扩展。在 6 时和 12 时，从目标日之前的最近 90 天中选择残差池；其中同月样本达到 14 天时优先使用同月样本。对历史日 $j < d$ ，用其对应发布时刻的名义预测构造联合残差

$$R_{j,t}^L = L_{j,t} - \tilde{L}_{j,t}, \quad R_{j,t}^{P,r} = P_{j,t} - \tilde{P}_{j,t}^{(r)}. \quad (18)$$

同一场景的负荷和光伏取自同一天，并保留整段曲线。每次调整使用 $S = 40$ 个等权情景，包含 1 个名义情景及 39 个有放回重采样情景；两次调整共抽样 78 个残差日索引，并不表示每次有 78 个场景。

不同情景共享调整量 x^a ，各自拥有充放电、弃电、储能与紧急补救变量。对剩余时段集合 $\mathcal{T}_r$ ，求解

$$\min \sum_{t \in \mathcal{T}_r} (1.5p_t x_t^a - p_t y_t) + \frac{1}{S} \sum_{s=1}^S \sum_{t \in \mathcal{T}_r} [5p_t e_t^s + \varepsilon(c_t^s + q_t^s)], \quad (19)$$

约束为各情景下的能量平衡、SOC 动态、容量与功率界限，以及式 (17) 和 y 的线性化约束。目标中的 $1/S$ 不可省略，否则情景数增加等价于人为提高紧急费权重。18 时采用名义剩余时段调整；这是一项计算与信息价值折中，并不意味着 18 时以后完全没有负荷不确定性。

场景内的补救轨迹仅服务于计划评价，其拥有的整段情景信息不等于实际执行可提前获知未来。候选扩展的实现费用也一律用逐槽因果执行回放计算：

$$C_3 = \sum_{d,t} [p_t x_{d,t}^a + 0.5p_t |x_{d,t}^a - x_{d,t}^0| + 5p_t e_{d,t}]. \quad (20)$$

正式方案则不求解式 (19)，在每个发布时刻只按最新组合名义预测调整剩余计划。这样既保留相同的信息结构和动态日末参考，又消除了场景抽样及完整情景补救带来的额外保守性。

7.5 整体表现与指定日期结果

正式主方案 334 天总费用 1347.1 万元，其中计划费 1290.5 万元、调整净额 25.2 万元、紧急费 31.4 万元。此处调整净额定义为 $\sum [g(x^a; x^0) - px^0]$ ，不是绝对偏差费本身。与官方预报无调整基线 1464.5 万元相比，费用降低 8.01%；与官方预报三点调整 1357.5 万元相比降低 0.76%。图 4 说明主要收益来自允许更新和调整，组合预测进一步小幅改善费用。表 9-12 同时列出零点计划与调整后结果，便于区分承诺、实际执行与最终结算。

表 9 问题三指定日期计划购电量与购电费

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00-10:10	0.00	0.00	0.00	0.00
12:00-12:10	567.21	0.00	419.53	966.15
14:00-14:10	0.00	0.00	0.00	0.00
16:00-16:10	618.29	187.98	586.23	811.78
18:00-18:10	706.57	0.00	787.06	735.24
20:00-20:10	0.00	0.00	0.00	0.00
全天计划电量/kWh	64357.79	40036.23	70612.27	96295.77
全天计划购电费/元	39979.30	22282.70	43935.46	61825.34

表 10 问题三指定日期调整购电量与购电费

时段或指标	3 月 20 日	6 月 21 日	9 月 23 日	12 月 21 日
10:00-10:10	0.00	0.00	0.00	0.00
12:00-12:10	567.21	0.00	419.53	999.84
14:00-14:10	0.00	0.00	0.00	0.00
16:00-16:10	520.24	160.44	543.94	822.58
18:00-18:10	706.57	0.00	787.06	735.24
20:00-20:10	0.00	0.00	0.00	0.00
调整后全天电量/kWh	62231.04	40260.15	68561.04	96287.52
买入及偏差费/元	40053.91	22874.05	43321.23	62209.20
紧急购电费/元	20.44	0.00	43.00	74.30
含紧急费总额/元	40074.34	22874.05	43364.23	62283.49

表 11 问题三指定日期储能充放电与首末电量 (kWh)

区间	3 月 20 日		6 月 21 日		9 月 23 日		12 月 21 日	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00-4:00	1292.66	122.05	447.73	200.31	1884.60	43.58	1956.07	276.70
4:00-8:00	774.90	5841.05	525.84	2870.08	424.36	6752.50	1608.84	1579.87
8:00-12:00	4175.67	2777.38	8988.07	0.00	5718.41	1896.62	5457.61	7155.69
12:00-16:00	6164.61	260.50	0.00	0.00	5429.52	500.05	7167.18	2237.06
16:00-20:00	233.47	5454.62	267.54	5481.91	188.48	5720.29	3.13	5687.61
20:00-24:00	4089.80	3017.89	8635.18	2570.34	8474.10	2944.14	8405.16	2848.38
0:00 电量	9137.12		5246.06		8851.11		8728.51	
24:00 电量	4780.15		9865.50		8917.34		8883.03	

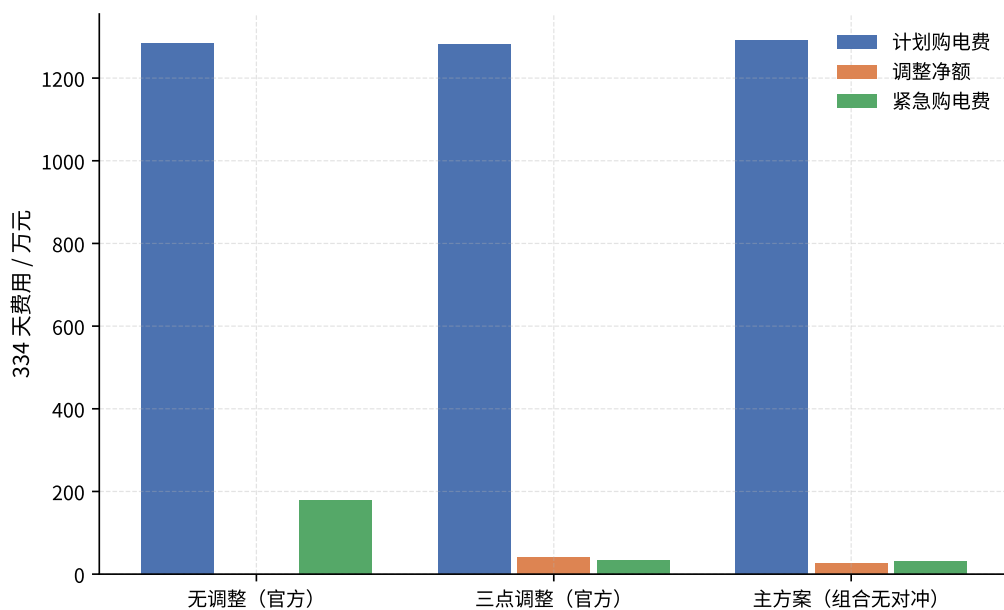


图 4 问题三不同信息与调整策略的费用分解

表 12 问题三指定日期全部紧急购电事件

日期	紧急购电时段	紧急购电量/kWh
3 月 20 日	20:30-20:40	2.9299
6 月 21 日	无	0.0000
9 月 23 日	20:20-20:30	6.3750
12 月 21 日	7:50-8:00	12.7909

7.6 消融检验与其他预报时刻

表 13 表明，组合预报在无对冲结构下比单一官方预报降低 10.4 万元。对冲扩展相对正式方案增加约 2.1 万元，且紧急费与紧急购电量均增加。进一步按预先划分区间核算，开发期对冲仅低 0.20 万元，而冻结验证期无对冲低 2.30 万元；因此这并非只按全年结果事后择优，留出证据同样不支持保留对冲。据此正式方案删除对冲，将其保留为负结果；未来只有在新增独立极端样本并验证 P95 或 CVaR 收益后才考虑恢复。

表 13 组合预报、对冲与信息一致性的消融结果

配置	总费用/万元	紧急费/万元
官方预报、三点调整、无对冲	1357.5	34.1
正式主方案：组合、三点调整、无对冲	1347.1	31.4
候选扩展：组合、三点调整及对冲	1349.2	33.6
计划组合、调整层仅官方且含对冲	1353.0	33.8

对于题目提出的其他发布时刻，现有附件只提供四次预报，无法可靠计算新增时刻的实际收益。可用“新增预报带来的期望紧急费节省，是否超过调整成本与获取成本”作为判据。早晨或中午天气突变时提高预报频率可能有价值，但不能由现有结果宣称某个未观测时刻最优。

本问结论：组合无对冲主方案费用 1347.1 万元，较官方无调整基线下降 8.01%；对冲在冻结验证期与全年均增费，故不进入正式策略。

八、 问题四：波动电价下的两层调度

8.1 给定电价 G 与历史预测 H

主口径 G 直接采用附件 4 的当日及次日电价替换两日规划中的固定价格，负荷和光伏信息范围保持不变。年末没有下一年度价格时，对第二日价格沿用最后可得日的曲线；第二日仅作为窗口延伸，不进入本年度报送费用。该模型评价在题目给定价格条件下能够达到的调度效果，不能将其解释为价格未知时的可实现收益。

为考察电价信息受限情况，扩展 H 采用三源预测

$$\hat{p}_{d,t} = v_{d,1}p_{d-1,t} + v_{d,2}p_{d-7,t} + v_{d,3}\bar{p}_t, \quad v_{d,i} \geq 0, \quad \sum_i v_{d,i} = 1. \quad (21)$$

候选权重按此前 7 天费用选择，再进行 $v_d = 0.1v_d^* + 0.9v_{d-1}$ 平滑。预测次日时，尚未实现的当日价格用最近已观测曲线替代。H 只改变优化时的价格输入，最终计划、偏差和紧急费用仍按实际交易价格结算。

8.2 无调整层与调整层结果

问题四分别沿用问题二与问题三的预测、SOC 与执行结构，仅替换电价。表 14 给出 334 天结果；G 两层费用分别为 1459.8、1410.3 万元，H 分别为 1465.9、1418.4 万元。有调整层的费用由买入、偏差与紧急费用构成，见图 5。日内调整降低了短缺量及

表 14 波动电价下的正式结果与历史价格扩展

指标	无日内调整层	有日内调整层
G 总费用/万元	1459.8	1410.3
G 紧急购电费/万元	92.8	40.5
G 紧急购电量/kWh	155951.8	77378.0
H 总费用/万元	1465.9	1418.4
H 相对 G 费用增幅	0.42%	0.57%

紧急费用；不能将与固定价问题的费用差直接解释成策略改进或退化，因为结算价格环

境已经变化。以 $I_p = (C_H - C_G)/C_G$ 度量条件电价信息价值，得到 0.42% 和 0.57%。该

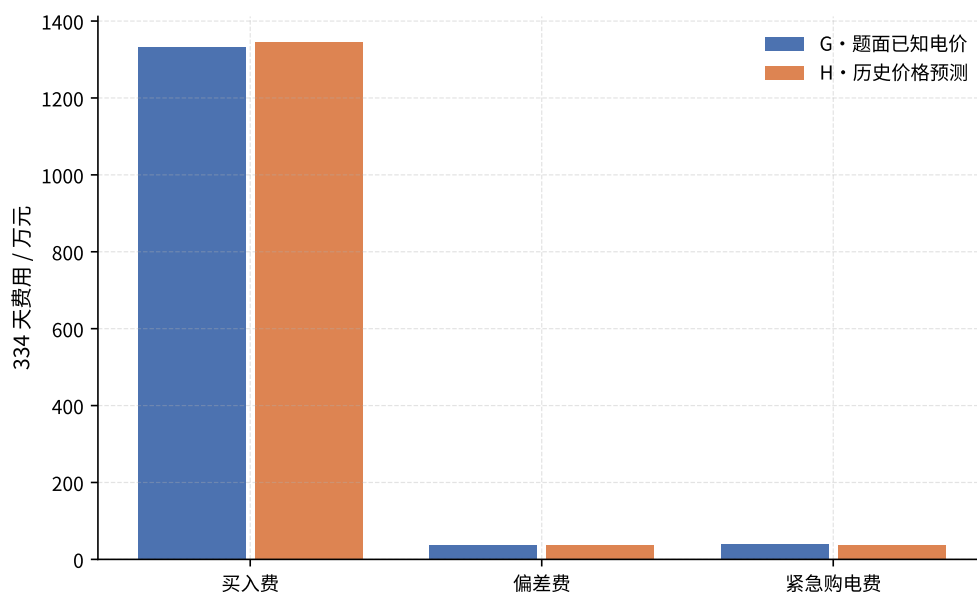


图 5 波动电价调整层在 G、H 两种信息条件下的费用分解

结果说明在这组数据及当前预测策略下，价格预测误差的额外成本较小；它不能推广为所有电价机制下的信息价值都很低。G 仍需面对未知的负荷与光伏，因此也不是所有不确定性消失后的理论最优下界。

8.3 物理可行性与结果校验

G 无调整层和调整层的功率平衡最大残差分别为 4.547×10^{-13} 与 3.411×10^{-13} kWh，SOC 递推最大残差均为 1.137×10^{-13} kWh；储电量落在 1200 至 10800 kWh，同槽同时充放次数为 0。调整层正式方案不使用场景池。完整计划、最终调整、分段充放电与紧急购电事件分别写入题目指定的两份结果表。

本问结论：波动价格条件下，无调整层费用 1459.8 万元，有调整层 1410.3 万元；历史电价预测扩展的额外费用分别为 0.42% 与 0.57%。

九、灵敏度分析与一致性检验

9.1 已删除对冲扩展的稳定性

为判断否定对冲是否只是某个随机种子的偶然结果，对已退出正式方案的候选扩展改变情景数、种子与历史池。表 15 取自独立灵敏度运行，参考配置为 1349.3 万元；本轮受控消融的对冲扩展为 1349.2 万元。这一 0.1 万元差异不影响其均高于正式无对冲方案 1347.1 万元的判断；由于旧灵敏度缺少未舍入逐日记录，不擅自将差异归因为舍入。40 情景在不同种子下费用变化较小，但各变体并未给出稳定优于 1347.1 万元正式方案的证据。短历史池略增费用，说明季节匹配与样本数量需平衡；该表只解释被删除

表 15 问题三灵敏度运行结果

配置	费用/万元	相对灵敏度参考值
40 情景, 种子 7 (参考)	1349.3	0
5 情景, 种子 7	1350.4	0.08%
10 情景, 种子 7/17/27	1350.6/1350.4/1351.5	0.10/0.08/0.16%
20 情景, 种子 7/17/27	1349.2/1350.1/1349.3	-0.01/0.06/0.00%
40 情景, 种子 17/27	1350.1/1349.0	0.06/-0.02%
历史池回看 30 天	1351.3	0.15%
历史池回看 60 天	1351.0	0.13%
组合权重 $\lambda = 0.5$	1349.1	-0.01%
组合权重 $\lambda = 0.9$	1357.7	0.62%

扩展的行为, 不再用于给正式策略选情景数。 $\lambda = 0.9$ 明显较差, 而 0.5 与 0.7 接近, 支持保留历史预测成分。

9.2 跨问题统一参数的代价

既有问题三参数搜索是在对冲候选结构下完成, 不能直接用于宣称无对冲正式方案的独立最优参数。本轮保持 $\lambda = 0.7, \kappa = 1.02, m = 25$ 不变, 只切换是否对冲, 以保证消融可归因; 其中 κ, m 仍沿用问题二开发期选定值, 便于跨问比较。冻结验证期仅用于判断对冲开关, 没有反过来重选连续参数, 从而避免重复利用验证集。

9.3 信息与约束的分层检验

实际执行按式 (11)–(12) 由当前槽构成, 结构上不读取未来实现值; 残差池检查每个索引严格小于目标日; EWMA 只吸收前一日及更早费用。这些检查验证操作层面的因果性, 但全局超参数使用 2 月至 6 月开发数据, 开发期自身仍是拟合内评价。另需区别场景 LP 的完整情景补救和在线因果控制, 不能把情景目标值直接当作可实现结算值。

五份结果文件均通过工作表名、日期、行数、计划与紧急量汇总检查。问题二和三的跨日 SOC 衔接最大误差为 0, 问题三计划费及计划量回读差为 0; 紧急事件表因小数输出与相邻事件合并产生微小勾稽差, 问题二约 2.300×10^{-4} kWh, 问题三约 1.083×10^{-3} kWh, 远低于实际调度电量尺度。

十、模型评价与推广

10.1 模型优点

模型把购电结算、储能动态与信息发布时间统一起来。两日滚动使储能能够跨日转移，避免每晚回到固定电量所产生的人为限制；逐槽因果执行使每一步控制可由当时可得信息实现。预测权重按历史费用更新，能响应非对称短缺惩罚；候选联合残差块保留负荷、光伏及日内相关结构，并经消融后未进入正式策略。不同问题共用预测参数、评价区间和初值，有利于比较日内调整与电价信息的影响。

实证结果同时给出主方案、对照、消融与时间留出性能。无对冲更便宜、验证期最优不一定等于开发期最优等负结果被保留，使复杂模块的边际贡献可以检验。

10.2 局限与改进方向

首先，单年数据难以覆盖更广的气象与电价状态，100 次重采样仅是条件压力评估。应增加独立年份或滚动多折验证，不能据本年度费用承诺未来节省。其次，评价区间从 2 月开始，并以 6000 kWh 重新初始化。进一步研究应从题目给定的 1 月 1 日初值连续运行，再提取 2 月至 12 月结果，以排除初始化方式的影响。

第三，历史权重成本表仍以单日规划作代理；日内调整也只优化当日剩余时段，并使用零点规划产生的动态日末参考。将权重标定和日内调整共同迁移到一致的多日滚动目标，可减少层间近似，但需要重新计算正式结果。两日窗口的最远端尚无剩余电量价值函数，也可能出现窗口末端耗尽倾向；延长窗口或使用经验证的终端价值是值得比较的改进方案。

第四，实际执行器只针对当前缺口充放电，未显式考虑未来价格机会成本。被删除的场景扩展采用整段情景补救，与在线执行存在信息差异；可进一步研究多阶段非预见性约束或价格感知的短窗实时控制。最后，当前场景对冲没有表现出冻结验证期或全年降本优势，是否改善新增独立情景下的尾部风险仍需检验；储能衰减和购电容量上限也可在扩展模型中加入。

10.3 结论

典型日优化购电费为 35126.95 元，较无储能节省 26.9%。在统一 334 天评价区间内，历史预测的两日滚动计划费用为 1395.7 万元；加入组合预报与三点调整后，无对冲主方案为 1347.1 万元。波动电价主口径下，两层费用分别为 1459.8 和 1410.3 万元。这些结果支持跨日储能和日内更新的应用价值，同时说明评价复杂调度方法时，应将可执行性、实现费用、尾部风险和假设分别检验。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于建模辅助、程序编写与调试、图表绘制、论文排版与验证，详细使用情况见支撑材料。

参考文献

[1] Mayne D Q, Rawlings J B, Rao C V, Scokaert P O M. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789–814. DOI: 10.1016/S0005-1098(99)00214-9.

[2] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142. DOI: 10.1007/s12532-017-0130-5.

[3] Rockafellar R T, Uryasev S. Optimization of conditional value-at-risk[J]. Journal of Risk, 2000, 2(3): 21–41. DOI: 10.21314/JOR.2000.038.

附录 A 复现说明与关键执行步骤

计算使用 Python、NumPy、SciPy 及 HiGHS，结果表通过 OpenPyXL 回读。运行环境提供 uv 与 requirements.txt 两种安装方式（详见支撑材料 README.txt）；正式 Q3 无场景随机性，对冲扩展种子为 7、蒙特卡洛种子为 42。复现时先进入随附 Python 工程目录。命令及结果对应关系见表 16，求解中的微小正则项不计入正文购电费用。复

表 16 核心复现入口

命令	对应内容
uv run python -m solve.q1	典型日调度
uv run python -m solve.q2_tune --result2	问题二正式结果
uv run python -m solve.q3	问题三正式结果
uv run python -m solve.q4_price_fit	历史电价成本标定
uv run python -m solve.q4 --result4-2	波动价格无调整层
uv run python -m solve.q4 --result4-3	波动价格调整层
uv run python -m solve.q3_ablation	正式方案与对冲扩展消融

现顺序应先形成历史成本表与权重，再执行问题二和问题三，最后重算电价标定与问题四。同一随机种子有助于比较方案，但线性规划的退化解、求解器版本和浮点阈值仍可能造成小规模数值差异。

```
for day in evaluation_days:
    weights = update_from_past_costs(day)
    forecast = predict_two_days(day, weights)
    plan = optimize_horizon(forecast, actual_soc)
    for slot in current_day:
        if forecast_is_released(slot):
            commitment = adjust_remaining_plan(
                released_forecast, actual_soc, plan)
            load, pv = observe_current_slot(slot)
            charge, discharge, emergency, spill = execute(
                commitment[slot], load, pv, actual_soc)
            actual_soc += eta * charge - discharge / eta
```

```
settle_day_and_export()
```

上述为算法逻辑说明，完整求解、时间映射和回读审计以随附源程序为准。论文指定日期表直接取自正式 Excel，计划量与调整量分别保留；紧急购电表列出全部对应事件，无事件日明确记为零。

附录 B AI 辅助说明

本次论文整理使用 AI 编码助手辅助核对已有结果、组织公式和段落、生成表格与 LaTeX 排版。数值依据为给定附件及随附求解程序输出，文献条目经公开来源核对。此说明记录本次辅助范围，不替代参赛者对题意、模型与最终提交材料的审阅。

附录 C 核心程序片段

以下给出论文方法对应的核心算法片段（线性规划矩阵构建、凸组合权重标定、裕度参数确认、逐槽因果执行）；完整可运行工程与运行环境见支撑材料 src/（支持 uv 与 requirements.txt 两种安装方式）。

一、线性规划矩阵构建

program/src/solve/core/lp.py :: plan_lp

```
1 def plan_lp(price_h, load_h, pv_h, e_start, e_terminal=None, eps=0.0):
2     """统一计划 LP: 返回完整解字典与 OptResult。
3
4     ``e_terminal=None``（统一口径默认）时规划窗口末端完全自由（不锚定终值）；
5     传入数值时只约束视野最远端 E(H)=e_terminal（供结构对照）。
6     中间每个自然日的末端 SOC 都是优化变量；每日只执行前 144 槽，
7     次日以真实末端 SOC 重新求解。
8     解字典键：x 购电、c 充、d 放、s 弃光、E 储电量（均为长度 H 数组）。
9     """
10    price_h = np.asarray(price_h, dtype=float)
11    load_h = np.asarray(load_h, dtype=float)
12    pv_h = np.asarray(pv_h, dtype=float)
13    H = len(price_h)
14    if len(load_h) != H or len(pv_h) != H:
15        raise ValueError("price/load/pv 的滚动视野长度必须一致")
16    n = 5 * H
17    c_obj = np.zeros(n)
18    # 目标函数：仅购电量 x 按电价计费；c/d 加 罚项抑制无意义充放
19    c_obj[0:H] = price_h
20    c_obj[H:3 * H] = eps
21
22    # ---- 等式约束行布局：[0, H) 平衡；[H, 2H) 动态；可选第 2H 行终端条件 ----
23    n_rows = 2 * H + (1 if e_terminal is not None else 0)
24    A_eq = lil_matrix((n_rows, n))
25    b_eq = np.zeros(n_rows)
26    for t in range(H):
27        # 功率平衡：+x_t + d_t - c_t - s_t = L_t - P_t
28        A_eq[t, 0 * H + t] = 1.0
29        A_eq[t, 1 * H + t] = -1.0
30        A_eq[t, 2 * H + t] = 1.0
31        A_eq[t, 3 * H + t] = -1.0
32        b_eq[t] = load_h[t] - pv_h[t]
33    for t in range(H):
34        # 储能动态：E_t - E_{t-1} - c_t + d_t = 0 (t=0 时右端为 e_start)
```

```

35     r = H + t
36     A_eq[r, 4 * H + t] = 1.0
37     if t:
38         A_eq[r, 4 * H + t - 1] = -1.0
39     A_eq[r, 1 * H + t] = -ETA
40     A_eq[r, 2 * H + t] = 1.0 / ETA
41     b_eq[r] = e_start if t == 0 else 0.0
42     # 终端条件 (可选): E_{H-1} = e_terminal (日循环/跨日循环锚定)
43     if e_terminal is not None:
44         A_eq[2 * H, 4 * H + H - 1] = 1.0
45         b_eq[2 * H] = float(e_terminal)
46
47     # 变量边界: x [0,∞), c,d [0,P], s [0,该槽可用光伏], E [E_min,E_max]
48     bounds = (
49         [(0.0, None)] * H
50         + [(0.0, P_MAX_E)] * H
51         + [(0.0, P_MAX_E)] * H
52         + [(0.0, float(v)) for v in pv_h]
53         + [(E_MIN, E_MAX)] * H
54     )
55     res = pm.optimize.solve_lp(c_obj, A_eq=csr_matrix(A_eq), b_eq=b_eq, bounds=bounds)
56     if not res.success:
57         raise RuntimeError(f"计划 LP 失败: {res.message}")
58     sol = {
59         "x": res.x[0:H],
60         "c": res.x[H:2 * H],
61         "d": res.x[2 * H:3 * H],
62         "s": res.x[3 * H:4 * H],
63         "E": res.x[4 * H:5 * H],
64     }
65     return sol, res

```

program/src/solve/core/lp.py :: adjust_day

```

1 def adjust_day(price, load_kwh, pv_fc_kwh, x_plan, e_init, t0, eps=EPS_TH, e_terminal=E0):
2     """Q3 调整 LP: 对 [t0, T) 重优化购电与储能, 偏差费线性化 (新预报视为已知)。
3
4     e_terminal: 段末 (当天 24:00) 目标储电量, 默认 E0 (日循环); 跨日模式传 2 日计划的当日末值。
5
6     变量块 (各 m = T-t0 个): 0=adj 购电, 1=c 充, 2=d 放, 3=s 弃光, 4=E 储电量, 5=y 结算基准
7     目标:  $\min \sum_{t \in [t0, T)} [1.5 \cdot p \cdot adj - p \cdot y] + \cdot \sum (c+d)$ 
8     其中  $y = \min(x_{plan}, adj)$  (系数为负  $\rightarrow$  约束  $y \leq x_{plan}$ ,  $y \leq adj$  下自动取等)
9     返回 [t0, T) 的 adj 序列。
10    """
11    m = T - t0
12    n = 6 * m
13
14    def I(b, i):
15        return b * m + i
16
17    c_obj = np.zeros(n)
18    # 目标系数: adj 记 1.5p, y 记 -p (等价于  $p \cdot adj + 0.5p|x_{plan}-adj|$ ), c/d 罚项
19    c_obj[0 * m:1 * m] = 1.5 * price[t0:]
20    c_obj[1 * m:2 * m] = eps
21    c_obj[2 * m:3 * m] = eps
22    c_obj[5 * m:6 * m] = -price[t0:]
23
24    # ---- 等式约束: [0,m) 平衡; [m,2m) 储能动态; 第 2m 行终端 ----
25    rows = 2 * m + 1
26    A_eq = lil_matrix((rows, n))
27    b_eq = np.zeros(rows)
28    for i in range(m):
29        t = t0 + i
30        A_eq[i, I(0, i)] = 1.0
31        A_eq[i, I(1, i)] = -1.0
32        A_eq[i, I(2, i)] = 1.0
33        A_eq[i, I(3, i)] = -1.0
34        b_eq[i] = load_kwh[t] - pv_fc_kwh[t]

```

```

35
36     r = m + i
37     A_eq[r, I(4, i)] = 1.0
38     if i:
39         A_eq[r, I(4, i - 1)] = -1.0
40         A_eq[r, I(1, i)] = -ETA
41         A_eq[r, I(2, i)] = 1.0 / ETA
42         b_eq[r] = e_init if i == 0 else 0.0
43     A_eq[2 * m, I(4, m - 1)] = 1.0
44     b_eq[2 * m] = e_terminal
45
46     # ---- 不等式: y_i x_plan_i (上次承诺); y_i - adj_i 0 ----
47     A_ub = lil_matrix((2 * m, n))
48     b_ub = np.zeros(2 * m)
49     for i in range(m):
50         t = t0 + i
51         A_ub[i, I(5, i)] = 1.0
52         b_ub[i] = x_plan[t]
53         A_ub[m + i, I(5, i)] = 1.0
54         A_ub[m + i, I(0, i)] = -1.0
55     bounds = (
56         [(0.0, None)] * m
57         + [(0.0, P_MAX_E)] * m
58         + [(0.0, P_MAX_E)] * m
59         + [(0.0, float(pv_fc_kwh[t0 + i])) for i in range(m)] # 弃光 预测光伏
60         + [(E_MIN, E_MAX)] * m
61         + [(0.0, None)] * m
62     )
63     res = pm.optimize.solve_lp(
64         c_obj, A_ub=csr_matrix(A_ub), b_ub=b_ub,
65         A_eq=csr_matrix(A_eq), b_eq=b_eq, bounds=bounds,
66     )
67     if not res.success:
68         raise RuntimeError(f"调整 LP 失败 (t0={t0}): {res.message}")
69     return res.x[0:m], res.x[4 * m:5 * m]

```

二、凸组合预测与参数标定

program/src/solve/models/weights.py :: softmax

```

1 def softmax(theta) -> np.ndarray:
2     """logit 向量 → 凸权重 (非负、和=1)"""
3     e = np.exp(theta - theta.max())
4     return e / e.sum()

```

program/src/solve/models/weights.py :: simplex_grid

```

1 def simplex_grid(step: float) -> np.ndarray:
2     """3 维凸权重网格 (非负、和=1), 组数 (n+1)(n+2)/2."""
3     n = int(round(1.0 / step))
4     return np.array(
5         [
6             (i / n, j / n, (n - i - j) / n)
7             for i in range(n + 1)
8             for j in range(n + 1 - i)
9         ]
10    )

```

program/src/solve/consistency.py :: ewma_weights_from_table

```

1 def ewma_weights_from_table(C, grid, hl: float = EWMA_HL,
2                             first_target: int = START_DAY,
3                             warm_from: int = WARM_FROM,

```

```

4         n_days: int = 365) -> dict:
5     """由 Q2E 成本表 ``C`` 生成逐日 EWMA 标定权重（无前视）。
6
7     目标日 d 的记分矩阵 M = 指数衰减加权 [warm_from, d-1] 的成本表行；
8     返回 ``{d: (w, u)}`` (w 为负荷权重、u 为光伏权重，均凸、和=1)。
9     """
10    decay = float(np.exp(-np.log(2.0) / hl))
11    M = None
12    for k in range(warm_from, first_target - 1):
13        M = C[k].copy() if M is None else decay * M + (1.0 - decay) * C[k]
14    out: dict[int, tuple[np.ndarray, np.ndarray]] = {}
15    for d in range(first_target, n_days):
16        M = C[d - 1].copy() if M is None else decay * M + (1.0 - decay) * C[d - 1]
17        i, j = np.unravel_index(np.argmin(M), M.shape)
18        out[d] = (grid[i].copy(), grid[j].copy())
19    return out

```

program/src/solve/q2_tune.py :: seqs_ewma

```

1 def seqs_ewma(model: AdaptiveWeightModel, hl: float):
2     """日费用行指数衰减加权（半衰期 hl 天）；hl 1e8 退化为历史均值。
3
4     统一实现见 ``consistency.ewma_weights_from_table``（唯一参数源）；
5     本函数仅做 {day: (w, u)} → 报送期列表的接口适配。
6     """
7     table = cs.ewma_weights_from_table(model.C, model.grid, hl=hl)
8     return [table[d] for d in REP]

```

program/src/solve/models/adaptive.py :: day_cost

```

1 def day_cost(self, d: int, w, u) -> float:
2     """第 d 天实际总费用：按预测制定计划（take-or-pay）+ 按实际执行 + 紧急购电。
3
4     执行口径与正式统一口径一致（``consistency.EXEC_POLICY="free"``）：
5     无段末硬目标，仅容量/功率约束。
6     """
7     l_kwh, p_kwh = self.forecast(w, u, d)
8     x, _E, _ = plan_day(self.price, l_kwh, p_kwh, EO, eps=self.EPS_PLAN)
9     ex = exec_segment_causal(
10         self.L[d] / 6.0, self.P[d] / 6.0, x, EO, None
11     )
12     return float(self.price @ x + EMERG_MULT * (self.price @ ex["e"]))

```

program/src/solve/models/adaptive.py :: _one_day

```

1 def _one_day(self, d: int):
2     """日历日 d：网格成本行 + 从网格最优出发的 Adam 精化权重（供次日使用）。"""
3     n = len(self.grid)
4     row = np.empty((n, n))
5     best = (np.inf, 0, 0)
6     for i, w in enumerate(self.grid):
7         for j, u in enumerate(self.grid):
8             c = self.day_cost(d, w, u)
9             row[i, j] = c
10            if c < best[0]:
11                best = (c, i, j)
12    theta0 = np.concatenate(
13        [np.log(self.grid[best[1]] + 1e-9), np.log(self.grid[best[2]] + 1e-9)]
14    )
15    if d + 1 < N_DAY: # 精化“用于次日”的权重：目标日 d+1，标定在日 d
16        theta = self.calibrate(d + 1, theta0, W=1)
17        w, u = softmax(theta[:3]), softmax(theta[3:])
18        polished = self.day_cost(d, w, u)
19        if polished < best[0]:
20            return d, row, theta, float(best[0]), float(polished)

```

```
21         return d, row, theta0, float(best[0]), float(best[0])
```

program/src/solve/models/adaptive.py :: calibrate

```
1  def calibrate(self, d: int, theta, W: int | None = None) -> np.ndarray:
2      """在窗口 [d-W, d-1] 上做 steps 步 Adam (中心差分梯度), 返回 6 维。
3
4      Adam 为纯 NumPy 实现, 超参数与 ``torch.optim.Adam`` 默认一致
5      (1=0.9、2=0.999、=1e-8), 避免引入深度学习框架依赖。
6      """
7      days = self.window_days(d, W)
8      th = np.asarray(theta, dtype=float).copy()
9      m = np.zeros(6) # 一阶矩 (梯度)
10     v = np.zeros(6) # 二阶矩 (梯度平方)
11     b1, b2, eps = 0.9, 0.999, 1e-8
12     for t in range(1, self.steps + 1):
13         grad = np.empty(6)
14         for i in range(6):
15             plus, minus = th.copy(), th.copy()
16             plus[i] += self.eps_grad
17             minus[i] -= self.eps_grad
18             grad[i] = (
19                 self.window_cost(plus, days) - self.window_cost(minus, days)
20             ) / (2 * self.eps_grad)
21             m = b1 * m + (1.0 - b1) * grad
22             v = b2 * v + (1.0 - b2) * grad * grad
23             m_hat = m / (1.0 - b1 ** t)
24             v_hat = v / (1.0 - b2 ** t)
25             th -= self.lr * m_hat / (np.sqrt(v_hat) + eps)
26     return th
```

三、裕度参数确认

program/src/solve/consistency.py :: kappa_load

```
1  def kappa_load(load_kw: np.ndarray, kappa: float = KAPPA) -> np.ndarray:
2      """负荷名义预测: •L (裁剪非负)。"""
3      return np.clip(np.asarray(load_kw, dtype=float) * kappa, 0.0, None)
```

program/src/solve/consistency.py :: margin_pv

```
1  def margin_pv(pv_kw: np.ndarray, margin: float = MARGIN) -> np.ndarray:
2      """光伏名义预测: max(P - m, 0)。"""
3      return np.clip(np.asarray(pv_kw, dtype=float) - margin, 0.0, None)
```

program/src/solve/q2_tune.py :: margin_refs

```
1  def margin_refs(model: AdaptiveWeightModel, seq, months) -> dict:
2      """由残差分布得到参考裕度: 全局/时段/小时/月/比例式/负荷系数。"""
3      e, lh = net_residuals(model, seq)
4      refs = {
5          "global": float(np.quantile(e, 0.80)),
6          "slot": np.quantile(e, 0.80, axis=0),
7          "hour": np.array([np.quantile(e[:, 6 * h:6 * (h + 1)], 0.80)
8                           for h in range(24)]),
9          "month": np.array([
10             np.quantile(e[months[REP] == m], 0.80) if (months[REP] == m).any() else 0.0
11             for m in range(1, 13)]),
12          "kappa": 1.0 + float(np.quantile((e / np.maximum(lh, 1.0)).ravel(), 0.80)),
13      }
14     ratios = []
```

```

15     for i, d in enumerate(REP):
16         w, u = seq[i]
17         l, p = model.forecast_kw(w, u, d)
18         mask = p > 500.0
19         if mask.any():
20             ratios.append(e[i][mask] / p[mask])
21     refs["rho"] = float(np.quantile(np.concatenate(ratios), 0.80))
22     refs["e"] = e
23     return refs

```

program/src/solve/q2_tune.py :: rolling_kappa

```

1  def rolling_kappa(model: AdaptiveWeightModel, seq, window: int = 60, q: float = 0.8):
2      """动态负荷系数：每日用过去 window 天残差比 e/L 的 q 分位数（无前视）。"""
3      e, lh = net_residuals(model, seq)
4      ratio = e / np.maximum(lh, 1.0)
5      kappa = np.ones(len(REP))
6      for i in range(len(REP)):
7          lo = max(0, i - window)
8          if i - lo >= 14:
9              kappa[i] = 1.0 + float(np.quantile(ratio[lo:i].ravel(), q))
10     return kappa

```

四、逐槽因果执行

program/src/solve/core/causal.py :: exec_segment_causal

```

1  def exec_segment_causal(load_kwh, pv_kwh, x_commit, e_start, e_end,
2                          no_purchase_charge: bool = False):
3      """逐槽因果执行：每槽只使用当前已实现的负荷和光伏，不读取未来实际值。
4
5      策略：先用储能吸收当前富余或填补当前缺口，再把下一时刻 SOC 投影到
6      "仍能在剩余时段到达 e_end"的可达区间；无法被储能覆盖的缺口记为紧急购电，
7      富余且无法消纳的部分记为弃电。
8
9      可选策略开关（2026-09-13 增设，供"段末目标机制"对照实验）：
10     - ``e_end=None``：完全自由末端（仅受容量/功率约束，不做可达性投影）；
11     - ``no_purchase_charge=True``：禁止"为充能而紧急购电"——SOC 不得被抬升到
12       自然平衡点之上，段末目标允许偏离（偏差在返回值中报告）。
13     参数均为长度 m 的 kWh/槽序列；返回 c/d/s/E/e（另附诊断量）。
14     """
15     load_kwh = np.asarray(load_kwh, dtype=float)
16     pv_kwh = np.asarray(pv_kwh, dtype=float)
17     x_commit = np.asarray(x_commit, dtype=float)
18     if not (load_kwh.ndim == pv_kwh.ndim == x_commit.ndim == 1):
19         raise ValueError("因果执行输入必须是一维序列")
20     if not (len(load_kwh) == len(pv_kwh) == len(x_commit)):
21         raise ValueError("因果执行的负荷、光伏与购电序列必须等长")
22     if not (E_MIN <= e_start <= E_MAX):
23         raise ValueError("因果执行的初始 SOC 超出安全区间")
24     if e_end is not None and not (E_MIN <= e_end <= E_MAX):
25         raise ValueError("因果执行的末端 SOC 超出安全区间")
26
27     m = len(load_kwh)
28     c = np.zeros(m)
29     d = np.zeros(m)
30     spill = np.zeros(m)
31     E = np.zeros(m)
32     emerg = np.zeros(m)
33     e_now = float(e_start)
34     max_up = ETA * P_MAX_E          # 单槽最大充电量 (kWh)
35     max_down = P_MAX_E / ETA        # 单槽最大放电量 (kWh)
36     tol = 1e-8
37

```

```

38     for i in range(m):
39         remain = m - i - 1
40         if e_end is None or no_purchase_charge:
41             # 自由末端（或禁止购电充电）：仅受容量与功率约束
42             lo = max(E_MIN, e_now - max_down)
43             hi = min(E_MAX, e_now + max_up)
44         else:
45             # E_next 既要能由当前 SOC 一步到达，也要能在剩余槽回到 e_end
46             lo = max(E_MIN, e_now - max_down, float(e_end) - remain * max_up)
47             hi = min(E_MAX, e_now + max_up, float(e_end) + remain * max_down)
48         if lo > hi + tol:
49             raise RuntimeError(
50                 f"终端 SOC 不可达: i={i}, interval=[{lo:.6f}, {hi:.6f}]"
51             )
52
53         net = load_kwh[i] - pv_kwh[i] - x_commit[i]
54         # 使当前槽供需平衡的自然 SOC 变化；若会破坏终端可达性则投影
55         delta_balance = -net / ETA if net >= 0.0 else -ETA * net
56         e_next = float(np.clip(e_now + delta_balance, lo, hi))
57         delta = e_next - e_now
58         if delta >= 0.0:
59             c[i] = delta / ETA
60         else:
61             d[i] = -delta * ETA
62
63         gap = load_kwh[i] + c[i] - pv_kwh[i] - x_commit[i] - d[i]
64         if gap >= 0.0:
65             emerg[i] = gap
66         else:
67             spill[i] = -gap
68         E[i] = e_next
69         e_now = e_next
70
71     term_err = abs(E[-1] - float(e_end)) if e_end is not None else 0.0
72     if e_end is not None and m and not no_purchase_charge and term_err > 1e-6:
73         raise RuntimeError(f"因果执行末端 SOC 偏差 {E[-1] - e_end:.3e} kWh")
74     return {"c": c, "d": d, "s": spill, "E": E, "e": emerg,
75            "terminal_error_kwh": float(term_err)}

```

附录 D 支撑材料文件列表

支撑材料压缩包的目录与文件应与表 17 逐项一致，用于支撑正文中的模型、结果与结论。其中，源程序应能够直接运行并复现主要数值结果；数据资料应附来源说明；较大篇幅的中间结果与论文图表放入 results/；人工智能工具使用情况另附 PDF 说明文件。

表 17 支撑材料压缩包文件列表

序号	文件或目录	内容与用途
1	src/	产生正文全部数值、表格与图件的完整可运行源程序（Python/uv 工程）及运行说明
2	data/	题目附件 1-5（含结果模板）及数据来源与口径说明
3	results/	五份正式结果表、参数扫描与中间输出、论文图表 PDF
4	AI 工具使用详情.pdf	AI 工具名称与版本、使用目的、关键交互记录、采纳内容及人工修改情况
5	README.txt	软件环境、依赖版本、随机种子与一键复现步骤